

GENERATIVE AI · RAG & AI Agents Deep Dive

RAG & AI Agents Deep Dive

LangChain · LlamaIndex · FAISS · ChromaDB · CrewAI · LangGraph

NIELIT Ropar · AI & ML With IIT Ropar · 2025–26

Module 1 Recap → RAG → Vector DBs → LangChain → AI Agents → CrewAI → LangGraph → Capstone

Module 1 Recap - What We Learned

Quick 5-minute refresher before we go deeper



Tokens & Embeddings

Text → tokens → high-dimensional vectors capturing semantic meaning. Similar meaning = closer vectors.



Transformers

Self-attention lets every word attend to all others in parallel. $Q \cdot K / \sqrt{d} \rightarrow \text{softmax} \rightarrow V$. Encoder-Decoder structure.



LLMs

Pretrained on trillions of tokens to predict next token. GPT, Llama, Claude, Gemma — all transformer-based.



Prompt Engineering

Zero-shot, Few-shot, Chain-of-Thought, Role prompting — guiding LLM output without any training.



RAG (Intro)

Retrieval-Augmented Generation: fetch relevant docs → inject into prompt → get grounded answers.



AI Agents (Intro)

LLM + Tools + Memory + Planning. ReAct loop: Observe → Think → Act. CrewAI, LangGraph as frameworks.

SECTION 01

RAG Deep Dive

Retrieval-Augmented Generation — Architecture, Chunking, Vector DBs & Advanced Patterns

Why RAG? The Problem with Bare LLMs

LLMs are powerful — but frozen in time, and prone to hallucination



Knowledge Cutoff

GPT-4 trained up to Apr 2023.
Asking about recent events = wrong answers.



Hallucination

LLMs confidently fabricate facts, citations, and statistics that sound plausible but are false.



No Private Data

Your company's PDFs, policies, and wikis don't exist inside the LLM at all.

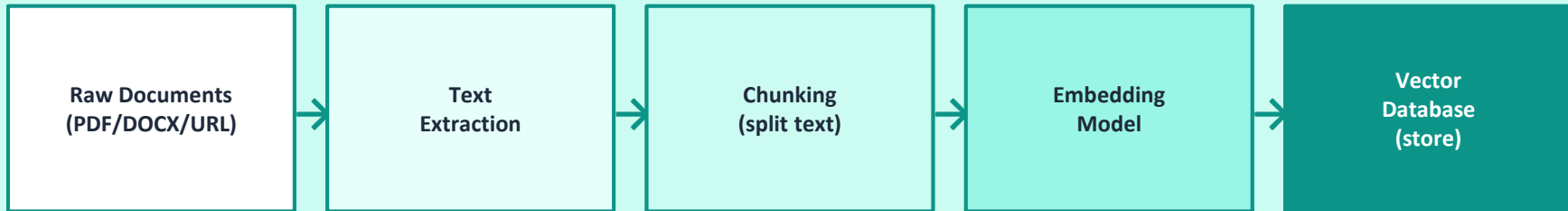


Fine-Tuning is Expensive

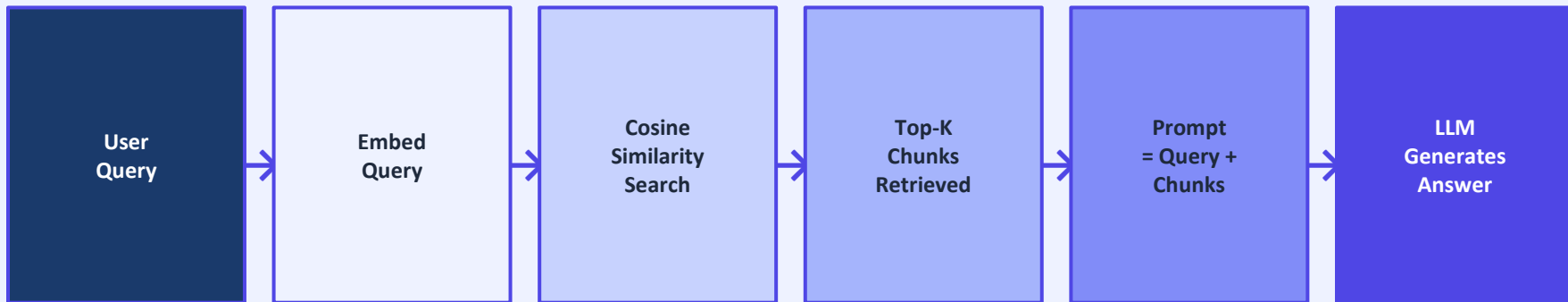
Updating model weights costs thousands of dollars, requires ML expertise, and still doesn't solve freshness.

RAG: Complete Architecture

OFFLINE INDEXING (run once)



ONLINE RETRIEVAL & GENERATION (every query)



Chunking Strategies - The Secret to Good RAG

How you split documents determines retrieval quality. Bad chunking = bad answers.

Fixed-Size Chunking

```
text_splitter = CharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=50  
)
```

- ✅ Simple, fast
- ⚠️ May split mid-sentence

Recursive Character Splitting

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    separators=["\n\n", "\n", " "]  
)
```

- ✅ Respects paragraph structure
- ⚠️ Slightly slower

Semantic Chunking

```
from langchain_experimental.text_splitter  
import SemanticChunker  
chunker = SemanticChunker(embeddings)
```

- ✅ Best quality, topic-aware
- ⚠️ Needs embedding model

Document-Aware Splitting

```
from langchain.document_loaders  
import PyPDFLoader  
# Splits by page boundaries
```

- ✅ Preserves document structure
- ⚠️ Works only for structured docs

Embedding Models - Choosing the Right One

The embedding model is the backbone of retrieval quality in RAG

Model	Org	Dims	MTEB Score	Best For
text-embedding-3-small	OpenAI	1536	62.3%	General purpose, production apps
text-embedding-3-large	OpenAI	3072	64.6%	High-accuracy enterprise RAG
all-MiniLM-L6-v2	SBERT (HuggingFace)	384	58.8%	Edge, offline, low-resource
bge-large-en-v1.5	BAAI (HuggingFace)	1024	63.9%	Best open-source for RAG
nomic-embed-text	Nomic AI	768	62.0%	Privacy-safe, local deployment



Tip: Use bge-large-en-v1.5 locally (free) or text-embedding-3-small via API for production.

SECTION 02

Vector Databases

FAISS · ChromaDB · Pinecone · Weaviate — Store, index & retrieve embeddings at scale

How Vector Databases Work

Traditional DBs match exact values. Vector DBs match by semantic closeness.

Traditional Database

```
WHERE name = 'RAG'
```

→ Exact string match only

→ 'Retrieval Augmented Gen' ≠ 'RAG'

Result: ❌ No match

VS

Vector Database

```
query = embed("What is RAG?")
```

→ Cosine similarity search

→ 'Retrieval Augmented Gen' ✅

→ 'RAG pipeline' ✅

→ 'Vector retrieval' ✅

Result: ✅ Top-K similar docs

Vector Database Showdown

Pick the right database for your project stage

Database	Type	Cost	Deploy	Best For
FAISS	In-memory library	Free	Local / Colab	Prototyping, research, offline RAG
ChromaDB	Embedded DB (local)	Free (open-source)	Local / Docker	Dev projects, classroom demos, small apps
Pinecone	Managed cloud service	Free tier → \$70+/mo	Cloud (serverless)	Production apps, enterprise RAG
Weaviate	Open-source / Cloud	Free / Cloud pricing	Self-host or cloud	Hybrid search, multi-modal

 **Tip: FAISS or ChromaDB (zero cost, no sign-up, run locally in Colab)**

FAISS - Code Walkthrough

```
# Install
# pip install faiss-cpu langchain langchain-openai

from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain.schema import Document

# Create documents
docs = [Document(page_content="RAG combines retrieval..."),
        Document(page_content="LangChain is a framework...")]

# Build FAISS index
embeddings = OpenAIEmbeddings()
db = FAISS.from_documents(docs, embeddings)

# Semantic search
results = db.similarity_search("What is RAG?", k=3)

# Save & Load
db.save_local("my_faiss_index")
db2 = FAISS.load_local("my_faiss_index", embeddings,
                      allow_dangerous_deserialization=True)
```

Key Concepts

OpenAIEmbeddings

Converts text to 1536-dim vector

from_documents()

Embeds all docs + builds HNSW index

similarity_search()

Embeds query, finds nearest vectors

k=3

Return top-3 most similar chunks

save_local()

Persist index to disk (avoid re-embedding)

Cosine similarity

Default distance metric in FAISS

ChromaDB — Persistent Local Vector Store

```
# pip install chromadb langchain

import chromadb
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

# Persistent storage – survives Colab restart!
embedding_fn = OpenAIEmbeddings()
vectordb = Chroma.from_documents(
    documents=docs,
    embedding=embedding_fn,
    persist_directory="./chroma_db"  # saved to disk
)

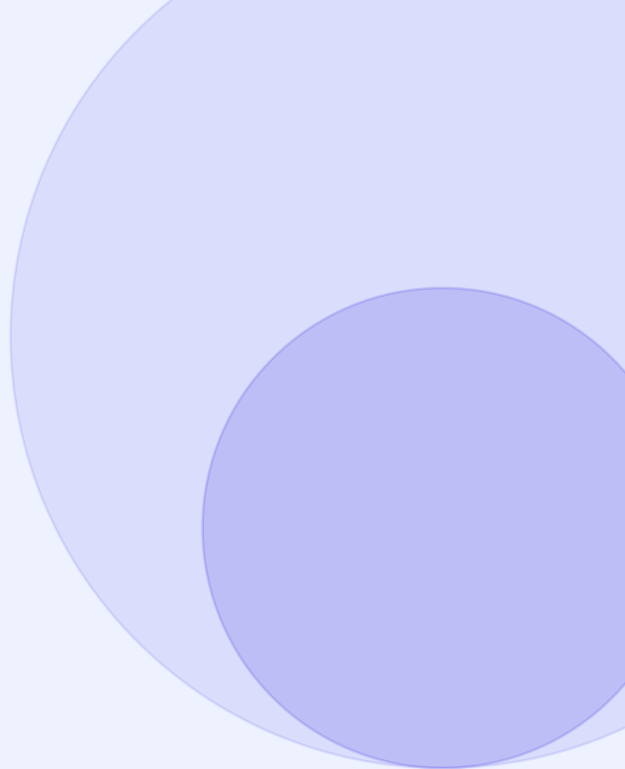
# Load existing collection
vectordb2 = Chroma(persist_directory="./chroma_db",
                    embedding_function=embedding_fn)

# MMR search (diversity-aware retrieval)
results = vectordb2.max_marginal_relevance_search(
    "What is chunking?", k=4, fetch_k=20)
```

SECTION 03

LangChain Deep Dive

Chains · Retrievers · Memory · LCEL — Build production RAG apps



LangChain: The AI Application Framework

Connects LLMs, tools, memory and data — so you build apps, not plumbing



LLMs / Chat Models

OpenAI, Anthropic, Ollama,
HuggingFace — unified interface



Prompt Templates

Structured, reusable prompt
blueprints with variable injection



Chains (LCEL)

Pipe operators | to compose LLM
workflows step-by-step



Document Loaders

PDF, DOCX, URL, YouTube,
Notion — 100+ loaders built-in



Text Splitters

Chunking strategies: fixed,
recursive, semantic, markdown-
aware



Retrievers

VectorStore, BM25, Ensemble —
fetch relevant context for RAG



Memory

ConversationBufferMemory,
SummaryMemory — multi-turn
chat



Agents & Tools

Calculator, search, code executor
— LLM decides what to call

LCEL — LangChain Expression Language

Modern LangChain uses the pipe | operator to compose chains declaratively

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

# Define components
llm = ChatOpenAI(model='gpt-4o-mini')
retriever = vectordb.as_retriever(search_kwargs={'k': 4})

prompt = ChatPromptTemplate.from_template("""
    Answer based on context only: {context}
    Question: {question}""")

# LCEL Chain – pipe operator composes the steps
rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

answer = rag_chain.invoke("What is retrieval-augmented generation?")
```



Key idea: Each step is a Runnable. The | operator chains them — output of one feeds input of next.

Conversational RAG — Adding Memory

Turn a one-shot Q&A into a full multi-turn chatbot with document memory

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationalRetrievalChain

memory = ConversationBufferMemory(
    memory_key='chat_history',
    return_messages=True
)

chain = ConversationalRetrievalChain.from_llm(
    llm=ChatOpenAI(model='gpt-4o-mini'),
    retriever=vectordb.as_retriever(),
    memory=memory
)

# Turn 1
r1 = chain({"question": "What is RAG?"})

# Turn 2 — remembers context!
r2 = chain({"question": "Give me an example of it"})
```

Memory Types

ConversationBufferMemory

Stores full history. Best for short chats.

ConversationSummaryMemory

Summarizes old turns. Saves tokens.

ConversationWindowMemory

Keeps last K messages only.

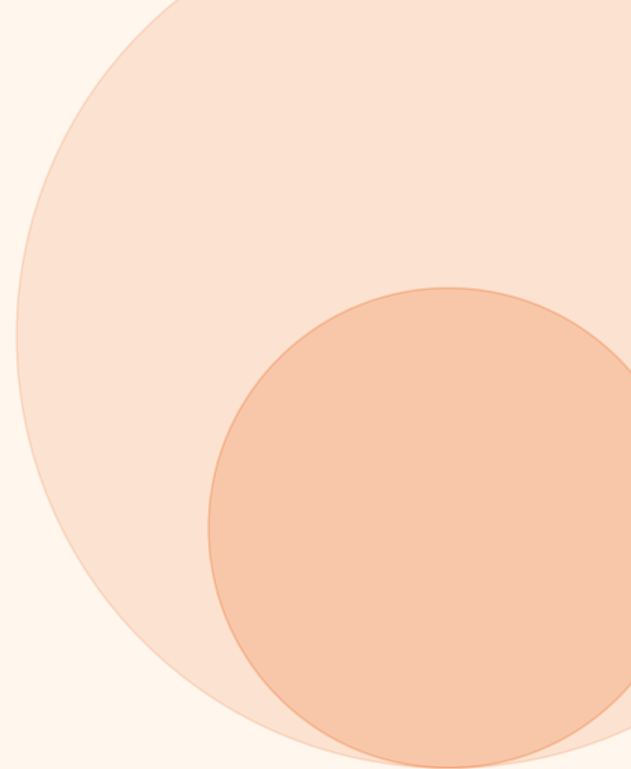
VectorStoreRetrieverMemory

Retrieves relevant past turns semantically.

SECTION 04

AI Agents Deep Dive

ReAct · Tools · Memory · Planning · Autonomous Task Execution



The ReAct Framework — How Agents Think

"Reasoning + Acting" — the cognitive loop behind modern AI agents

Example Trace	
User:	What is the capital of France and its population?
Think:	I need to search for France's capital first.
Act:	search('capital of France')
Obs:	Paris is the capital of France.
Think:	Now I need the population.
Act:	search('Paris population 2024')
Obs:	2.1 million (city proper)
Ans:	Paris is the capital. Population ≈ 2.1M.



LangChain Agents — Tools & Code

Agents decide which tool to call based on the user query

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain.tools import Tool
from langchain_community.tools import DuckDuckGoSearchRun
from langchain import hub

# Define tools
search = DuckDuckGoSearchRun()

def calculate(expr: str) -> str:
    """Evaluate math expression"""
    return str(eval(expr))

tools = [
    Tool(name="Search", func=search.run,
        description="Search the web for info"),
    Tool(name="Calculator", func=calculate,
        description="Solve math problems"),
]

llm = ChatOpenAI(model='gpt-4o-mini')
prompt = hub.pull('hwchase17/react')
agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
result = executor.invoke({"input": "What is 25 * 4 + search for LangChain?"})
```

Built-in LangChain Tools

DuckDuckGoSearch

Web search, no API key

WikipediaQueryRun

Wikipedia articles

PythonREPLTool

Execute Python code

RequestsGetTool

HTTP GET requests

SQLDatabaseTool

Query SQL databases

FileManagementTool

Read/write files

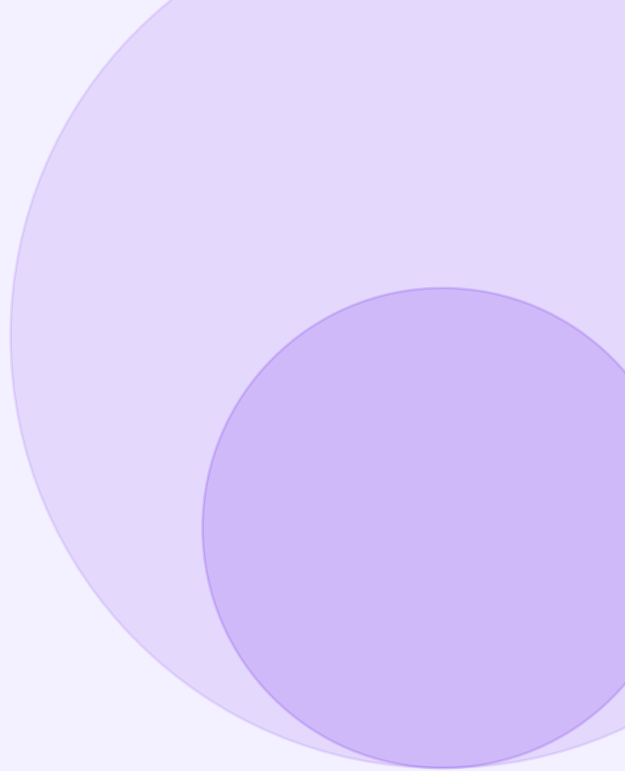
ShellTool

Run shell commands

SECTION 05

CrewAI — Multi-Agent Orchestration

Roles · Tasks · Crew · Process — Build teams of AI agents



CrewAI: AI Agents Working as a Team

Inspired by real-world team dynamics — each agent has a role, goal, and backstory



Agent

An AI with a specific role, goal, and backstory.
Defines personality + expertise.
Example: 'Senior Research Analyst at a tech firm'



Task

A specific unit of work assigned to an Agent.
Has description, expected_output, and assigned agent.
Can pass output to the next task.



Crew

A collection of Agents + Tasks.
Orchestrates who does what.
Runs tasks sequentially or in parallel.



Process

Execution strategy:
Sequential → tasks run one after another
Hierarchical → manager delegates to sub-agents

CrewAI — Full Code Example

```
from crewai import Agent, Task, Crew, Process
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini')

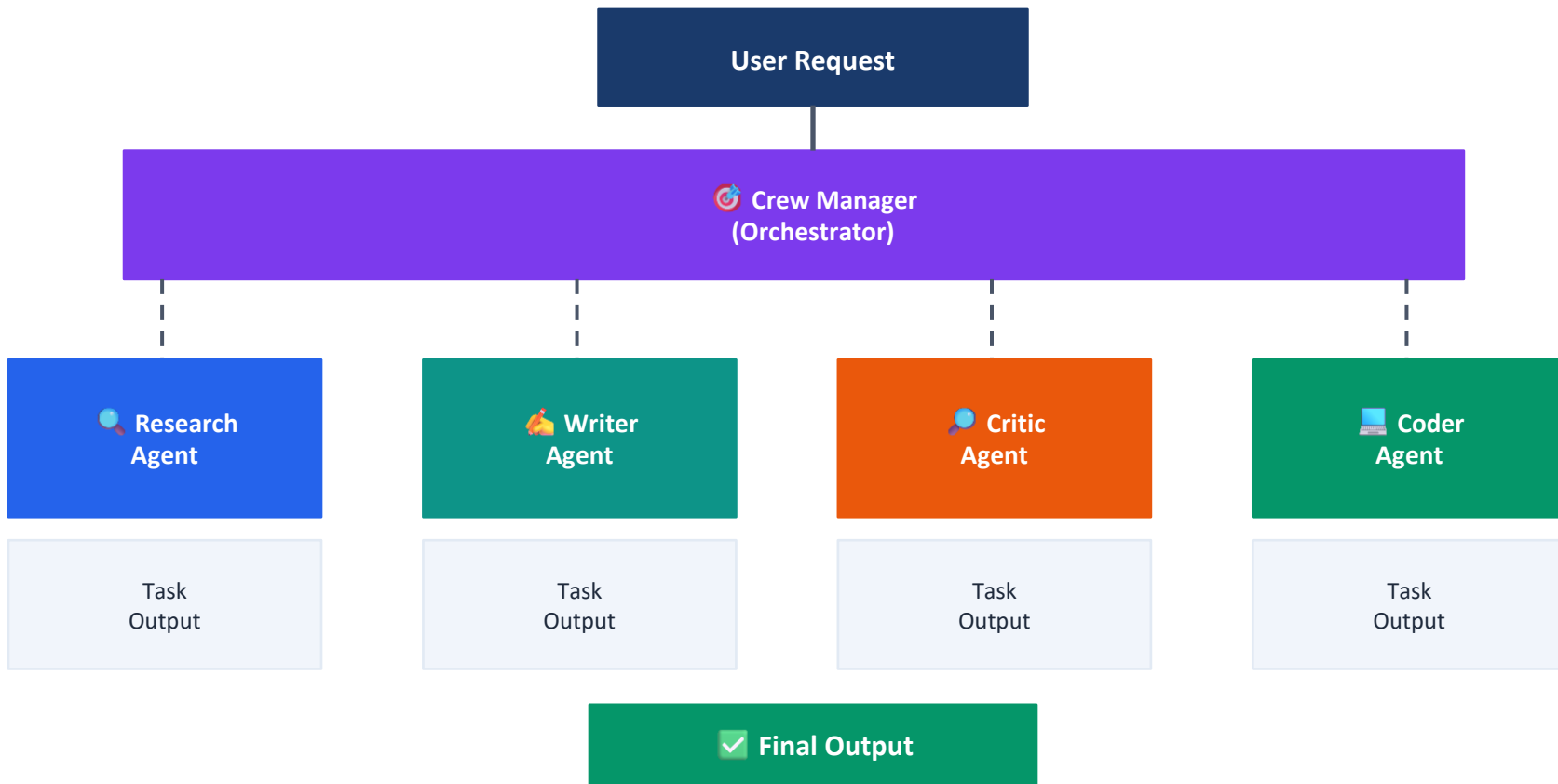
# 1. Define Agents
researcher = Agent(
    role='Senior Research Analyst',
    goal='Find accurate, up-to-date information on AI trends',
    backstory='10 years in AI research at top tech firms',
    llm=llm, verbose=True
)
writer = Agent(
    role='Technical Writer',
    goal='Write engaging, accurate blog posts about AI',
    backstory='Former IEEE editor, specialises in accessible AI writing',
    llm=llm, verbose=True
)

# 2. Define Tasks
research_task = Task(description='Research top 5 AI trends in 2025',
    expected_output='Bullet list with sources', agent=researcher)
write_task = Task(description='Write a 500-word blog post using research',
    expected_output='Full blog post', agent=writer)

# 3. Assemble & Run Crew
crew = Crew(agents=[researcher, writer],
    tasks=[research_task, write_task], process=Process.sequential)
result = crew.kickoff(inputs={'topic': 'Generative AI in Healthcare'})
```

CrewAI Workflow Visual

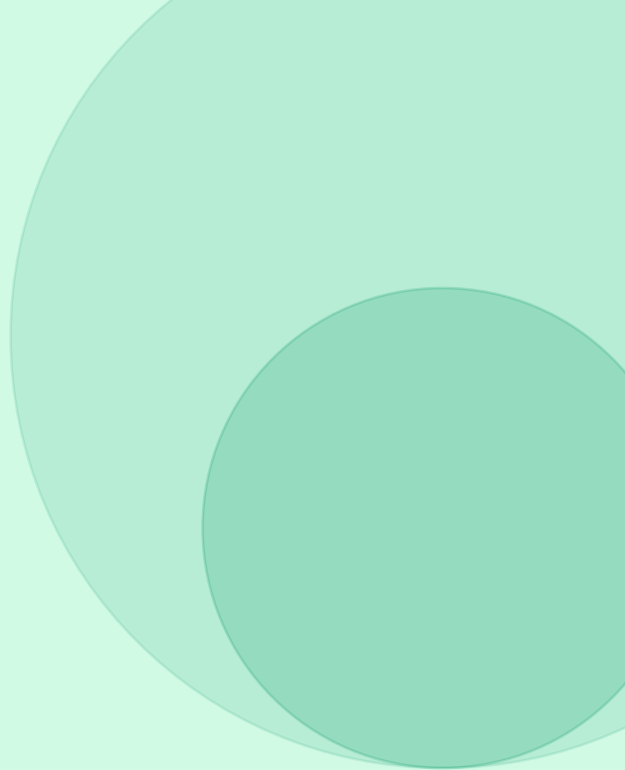
How agents collaborate to complete complex, multi-step tasks



SECTION 06

LangGraph — Stateful Agent Workflows

Nodes · Edges · State · Conditional routing — Graph-based agent control



LangGraph: Why Do We Need It?

LangChain Chains are linear. Real agents need loops, branches, and state.

LangChain Chains

Linear: $A \rightarrow B \rightarrow C \rightarrow \text{Done}$

No loops or cycles

No conditional branching

No persistent state between steps

Good for: simple Q&A, summarization

✗ Cannot: retry on error

✗ Cannot: loop until done

✗ Cannot: pause and wait for human

VS

LangGraph

Graph: any node $\rightarrow$ any node

Supports cycles and loops

Conditional edges (if/else routing)

Persistent state across all nodes

Good for: complex, multi-step agents

✓ Can: retry on error

✓ Can: loop until condition met

✓ Can: pause for human approval

LangGraph Core Concepts

Think of it as a directed graph where each node is a function

◆ State

A shared dictionary (TypedDict) that flows through the entire graph. Each node reads and updates it.

```
class State(TypedDict):  
    messages: list  
    next_step: str
```

⬡ Node

A Python function that takes State as input and returns an updated State. Can call LLMs, tools, or any logic.

```
def researcher(state: State):  
    # do work  
    return {"messages": [...]}
```

➡ Edge

Connection between nodes. Can be normal (always go there) or conditional (choose path based on state value).

```
graph.add_edge("research", "write")  
graph.add_conditional_edges(  
    "check", router_fn)
```

▣ Entry/End Point

Graph starts at set_entry_point. Ends at END node (from langgraph.graph). Can loop back too.

```
graph.set_entry_point("start")  
graph.add_edge("final", END)
```

LangGraph — Research + Write Agent

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class State(TypedDict): # shared state across all nodes
    query: str; research: str; draft: str; revision_count: int

def research_node(state: State) -> State:
    result = llm.invoke(f'Research: {state["query"]}')}
    return {**state, 'research': result.content}

def write_node(state: State) -> State:
    prompt = f'Write blog using: {state["research"]}'
    return {**state, 'draft': llm.invoke(prompt).content,
            'revision_count': state['revision_count']+1}

def should_revise(state: State) -> str:
    # Conditional edge: revise up to 2 times
    return 'revise' if state['revision_count'] < 2 else 'done'

graph = StateGraph(State)
graph.add_node('research', research_node)
graph.add_node('write', write_node)
graph.set_entry_point('research')
graph.add_edge('research', 'write')
graph.add_conditional_edges('write', should_revise,
    {'revise': 'write', 'done': END})
app = graph.compile()
result = app.invoke({'query': 'AI trends 2025', 'revision_count': 0})
```

CrewAI vs LangGraph — When to Use Which?

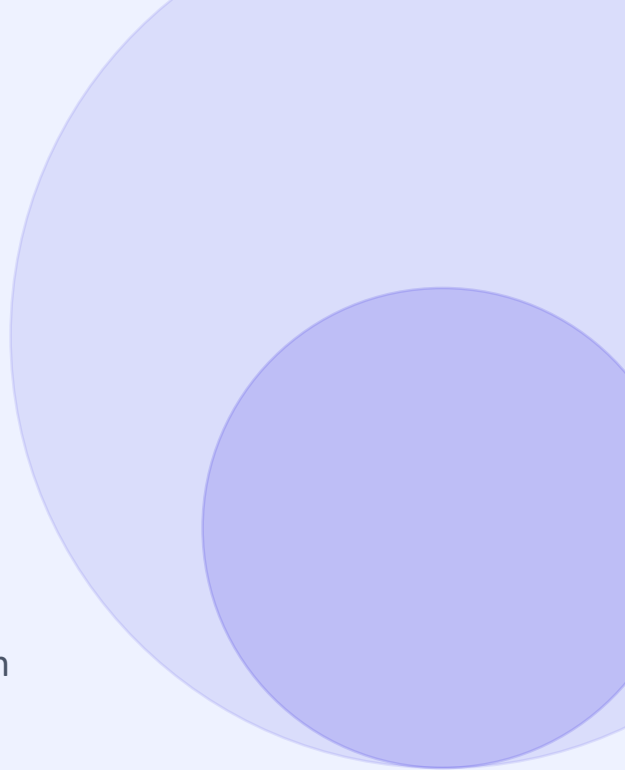
Both are great — they solve different problems

Feature	CrewAI	LangGraph
Mental Model	Team of human-like roles	Programmable state machine
Best For	High-level multi-agent tasks	Fine-grained agent control
Loops / Cycles	Limited	Native support
State Management	Passed via task output	Explicit typed StateGraph
Conditional Routing	Via task dependencies	<code>add_conditional_edges()</code>
Human-in-the-Loop	Limited	<code>interrupt_before/after</code>
Learning Curve	Easy — 20 lines to start	Moderate — more code
Streaming Support	Partial	Full streaming support
Production Use	✅ Great for pipelines	✅ Great for agents

SECTION 07

Capstone Projects

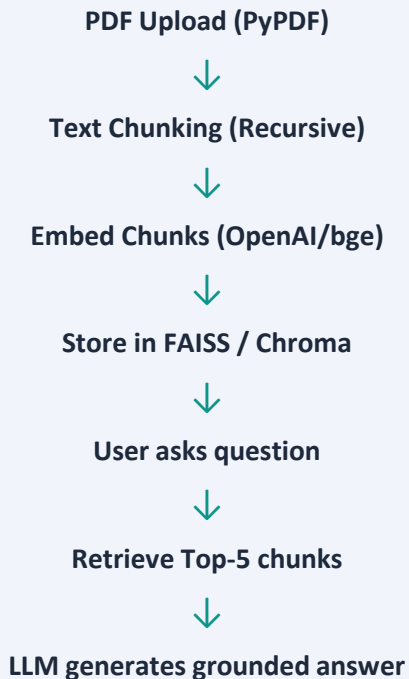
Build real systems using RAG + Agents + LangChain + CrewAI + LangGraph



Project 1 — PDF Q&A Chatbot (RAG)

 *Beginner · Build in 1–2 hours on Google Colab*

Architecture



Tech Stack & Features

Frontend:	Streamlit or Gradio
PDF Loading:	LangChain PyPDFLoader
Chunking:	RecursiveCharacterTextSplitter
Embeddings:	OpenAI / bge-large (free)
Vector DB:	FAISS (local, free)
LLM:	GPT-4o-mini / Llama3 (Ollama)
Memory:	ConversationBufferMemory
Deploy:	HuggingFace Spaces (free)

Project 2 — CrewAI Research & Report Agent

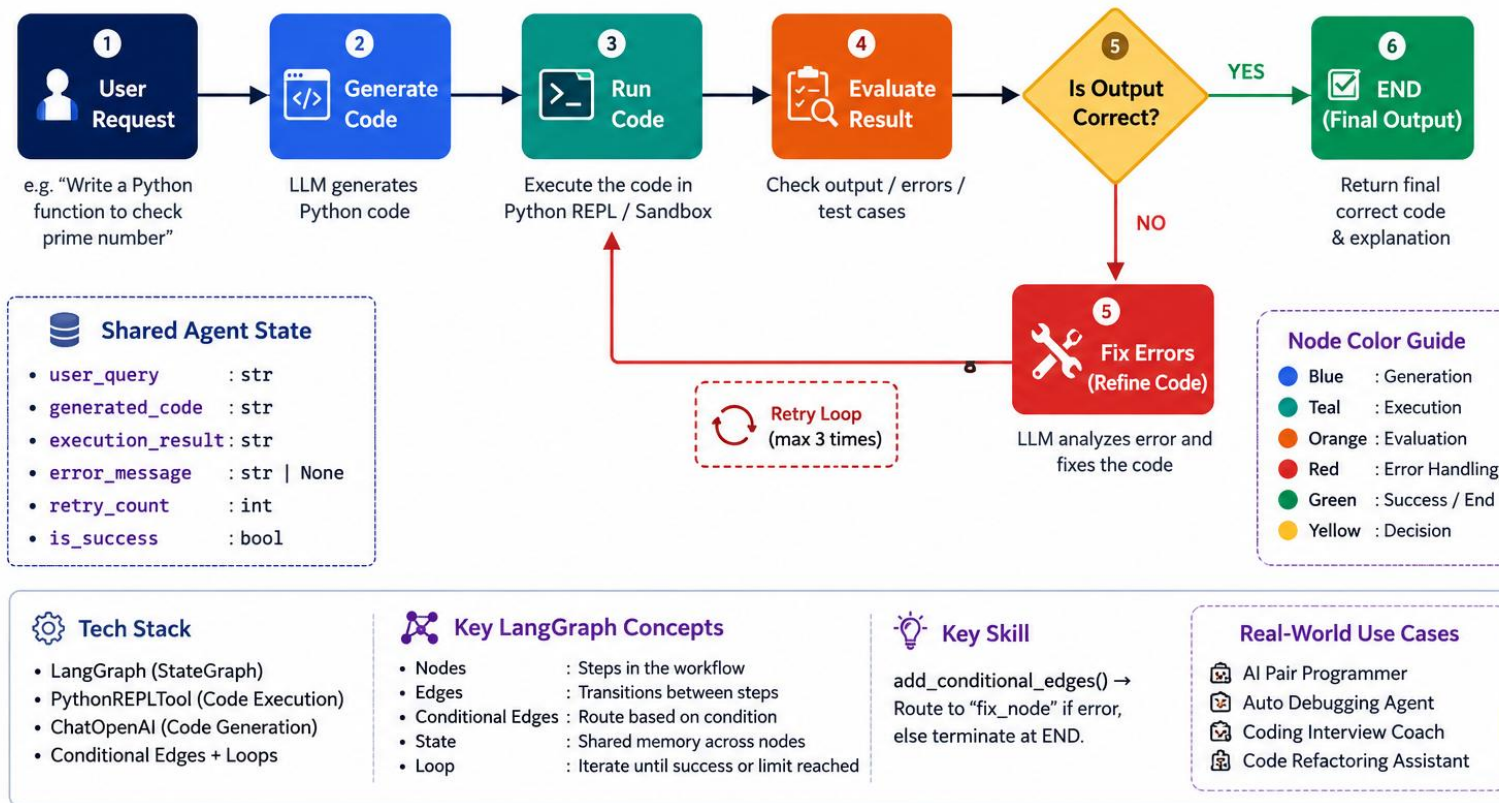
 *Intermediate · Multi-agent pipeline that researches, writes & critiques*



Tech: CrewAI + LangChain + DuckDuckGoSearch + OpenAI + Python markdown output

Project 3 — LangGraph Self-Correcting Code Agent

 **Advanced** · Agent that writes, tests, fixes, and iterates on Python code

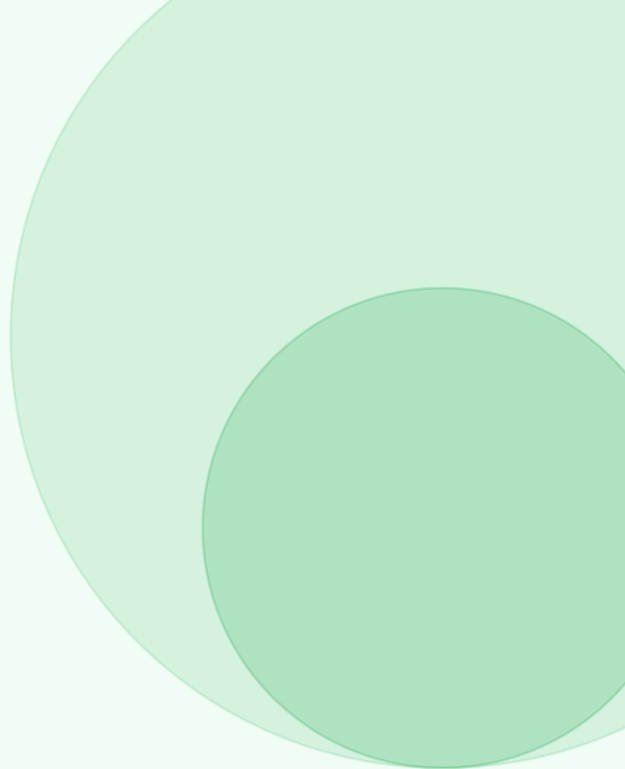


★ This agent demonstrates autonomous reasoning, error detection, self-correction, and iterative improvement.

SECTION 08

Advanced RAG Patterns

HyDE · Self-Query · Re-ranking · Hybrid Search · Agentic RAG



Beyond Basic RAG — Advanced Patterns

These techniques close the gap between prototype and production

HyDE (Hypothetical Document Embeddings)

Generate a hypothetical answer first, embed that, then retrieve. Improves retrieval for vague queries.

```
hyp = llm.invoke("Write a doc  
about: "+query)  
results =  
db.similarity_search(hyp)
```

Self-Query Retriever

LLM generates structured metadata filters from natural language query, improving precision.

```
retriever =  
SelfQueryRetriever.from_llm(  
    llm, vectordb,  
    doc_description, attrs)
```

Re-Ranking (Cross-Encoder)

Retrieve 20 chunks with fast bi-encoder, then re-rank top-5 with slower but accurate cross-encoder.

```
from langchain.retrievers import  
ContextualCompressionRetriever  
from  
langchain.retrievers.document_co  
mpressors import  
CrossEncoderReranker
```

Hybrid Search (Dense + Sparse)

Combine semantic (FAISS/Chroma) with keyword (BM25) search. Best of both worlds.

```
from langchain.retrievers import  
EnsembleRetriever  
hybrid = EnsembleRetriever(  
    retrievers=[bm25, faiss],  
    weights=[0.4,0.6])
```

Agentic RAG

Agent decides whether to retrieve, what to retrieve, when to stop. Uses tools to query multiple knowledge bases.

```
# Agent with retriever as a tool  
retriever_tool =  
create_retriever_tool(  
    retriever, "search_docs",  
    "Search PDFs")
```

RAPTOR (Multi-level Summarization)

Recursively cluster and summarize chunks, creating a tree structure for multi-scale retrieval.

```
# Cluster chunks → summarize  
clusters  
# Store summaries + originals  
# Retrieve from any level of  
tree
```

Module 2 — Key Takeaways

What you should be able to build after today

01

RAG is the #1 technique to make LLMs reliable for domain-specific use cases — no hallucinations on known docs.

02

Chunking strategy + embedding model choice are the two biggest levers for RAG quality. Start with bge-large.

03

LangChain = the standard toolkit: loaders, splitters, vectorstores, LCEL chains, memory, agents. Learn LCEL.

04

FAISS for local/free, ChromaDB for persistence, Pinecone/Weaviate for production at scale.

05

CrewAI = human-like team of agents for high-level tasks. Easy to start. Great for report/research pipelines.

06

LangGraph = programmable state machine for precise agent control. Use when you need loops, retries, and branches.

Your Learning Path After Module 2

What to do this week to lock in today's knowledge

Tonight

Run the FAISS code walkthrough in Colab. Upload any PDF. Get it working.

Day 3

Build the PDF chatbot with Streamlit UI. Add conversation memory.

Day 4

Swap FAISS for **ChromaDB**. Try bge-large embeddings locally with Ollama.

Day 5

Build CrewAI research agent. Give it a topic you care about.

Day 6

Replicate the LangGraph self-correcting code agent. Add a 3rd node.

Week 2

Deploy your RAG chatbot on HuggingFace Spaces. Add it to your GitHub portfolio.

 **Goal: By Day 7 you should have a working RAG chatbot deployed and a multi-agent pipeline running locally.**

Day 2 Resources & References

Everything you need to go deeper

Must-Read

- RAG Survey — arXiv 2312.10997
- LangChain Docs: python.langchain.com
- LangGraph Docs: langchain-ai.github.io
- CrewAI Docs: docs.crewai.com
- FAISS Docs: faiss.ai
- Self-RAG paper — arXiv 2310.11511

Tutorials

- LangChain YT: @LangChain
- Sam Witteveen RAG series (YouTube)
- Greg Kamradt (YouTube) — Chunking
- DeepLearning.AI: LangChain for LLMs
- DeepLearning.AI: Building Agentic RAG
- Colab: tinyurl.com/rag-colab-demo

Tools to Install

- `pip install langchain langchain-openai`
- `pip install langchain-community faiss-cpu`
- `pip install chromadb crewai langgraph`
- `pip install sentence-transformers`
- `pip install streamlit gradio pypdf`
- `ollama pull llama3` (local LLM, free)

Q & A

Questions & Discussion

Ask anything about RAG, LangChain, Vector DBs, CrewAI, LangGraph, or AI Agents

lovnishverma.in · @lovnishverma · NIELIT Ropar

Thank You

RAG & AI Agents Deep Dive

 **Portfolio:** lovnishverma.in

 **GitHub:** github.com/lovnishverma

 **Hugging Face:** huggingface.co/lovnishverma

 **Email:** princelv84@gmail.com

 **NIELIT Ropar:** Project Engineer

